

Improving Counting Sort Algorithm Via Data Locality

Mahmud, Haque & Choudbury - ACMSE 2022

Group 17 : Anna Curry - 24304011 | Carrie Savage - 24354821 | Jewel Hicks - 24221876 | Julie Seery - 24415002



Introduction

Sorting algorithms are foundational to computer science, used in database indexing to network scheduling. While theoretical complexities are well understood, real world performance is heavily influenced by CPU cache behaviour.

Counting Sort

- Counting Sort has a time complexity of $O(n + r)$
- Random input causes non-linear access to `count[]` and `output[]`, producing cache misses

Key Findings

- Sorted input runs 2-3x faster than random input for identical n & r
- Proposed Solution** : Cache friendly hybrid sorting algorithm combining Counting Sort with a modified Quick Sort preprocessing step

Proposed Method : Cache Friendly Hybrid Algorithm

Two Phase Hybrid Approach

Phase 1 : Preprocessing

Modified Quicksort partitions the input array into sub arrays, satisfying the CPU cache constraint. Sub arrays are not fully sorted, only partitioned.

Phase 2 : Sorting

Counting Sort is applied independently to each sub array. Sub arrays are ordered relative to each other, so output is globally sorted with cache friendly access patterns.

Stopping Condition : $(\text{maxVal} - \text{minVal} + \text{high} - \text{low}) \leq C$

Limitations

- Sacrifices stability due to Quicksort preprocessing step
- Threshold C is hardware dependent - not universal
- Additional preprocessing introduces overhead for small inputs
- Does not work for negative numbers

Modified Quicksort - Algorithm 2

```
void quicksortModified(int[] arr, int low, int high, int maxValue,
int minValue) {
    if (maxValue == minValue) {
        return;
    }
    while ((low < high) && (maxValue-minValue+high-low) > C) {
        int pivot = partition(arr, low, high);
        int midVal = arr[pivot];
        quicksortModified(arr, low, pivot-1, midVal, minValue);
        quicksortModified(arr, pivot+1, high, maxVal, midVal);
        return;
    }
}
```

Implementation Details

countingSort()

- Classic counting sort algorithm
- Uses `getMax()` to size `count[]` array
- Computes prefix sums
- Fills `output[]` in reverse order for stability
- Copies back to array

quicksortModified()

- Core of the proposal
- Checks stopping condition per call
- If exceeded, partition and recurses with updated min/max bounds

partition()

- Pivot selection using median of three values
- Hoare scheme used to partition

countingSortByPartition()

- Applies counting sort per partition
- Scans forward tracking `partMin/partMax`
- when adding next element exceeds cache threshold, calls `countingsortSegment()` on that segment

countingSortSegement()

- Counting sort on a single cache sized segment
- Writes sorted values back in a single pass

Key Challenges Encountered

Incomplete Sort Misunderstanding

`quicksortModified()` is an intentionally incomplete sort. Initial instinct was that it would fully sort the array. Discovered after large array testing that partitions are not individually sorted, the counting sort algorithm must be applied per partition not just once on the full array

Infinite Recursion on Duplicate Values

When `maxValue == minValue`, the stopping condition depends on `high - low`, so large equal element sub arrays exceed threshold C . Fixed by returning immediately when `maxVal == minVal`

Full Array VS Per Partition Counting Sort

Initially called `countingSort()` once on the entire post processed array. This produced correct results but missed the core idea. Instead we implemented `countingSortByPartitions()` to apply Counting Sort independently to each cach sized partition.

Experimental Setup

Platform : Apple M1 MacBook Air (192KB L1 instruction, 128KB L1 data, 12MB shared L2 cache per core)

Language : Java - `System.nanoTime()` for timing

Methodology : 5 trials per case, fixed random seed (21)

Input : Uniformly distributed random integers in $[0, r-1]$

Table 1 - Random VS Sorted Input

Expected : Sorted input should run 2-3x faster due to cache locality

n = r	Random (ms)	Sorted (ms)	Speedup
1,000,000	8.353	6.195	1.35x
2,000,000	10.986	6.309	1.73x
3,000,000	25.222	10.113	2.49x

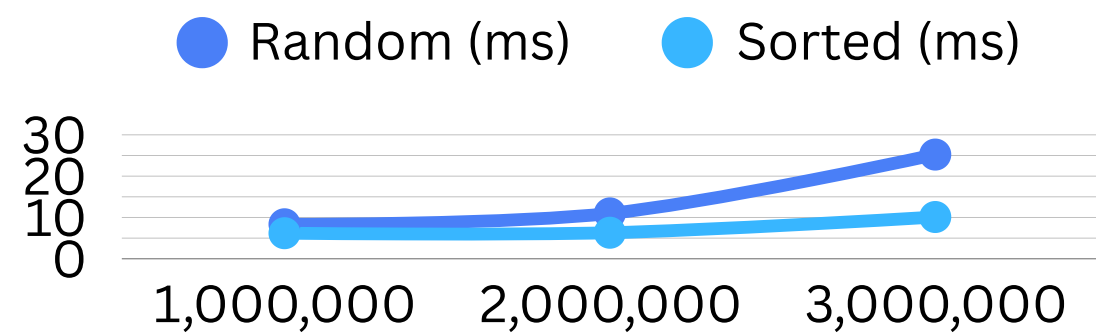


Table 2 - Hybrid VS Plain Counting Sort

Expected : Preprocessing + Counting Sort < Plain Counting Sort

r	n	Preprocess	Count Sort	Total Hybrid	Plain
1M	1,000	0.053	0.646	0.7	0.86
1M	2,000	0.053	0.308	0.361	0.896
1M	3,000	0.079	0.424	0.503	0.89

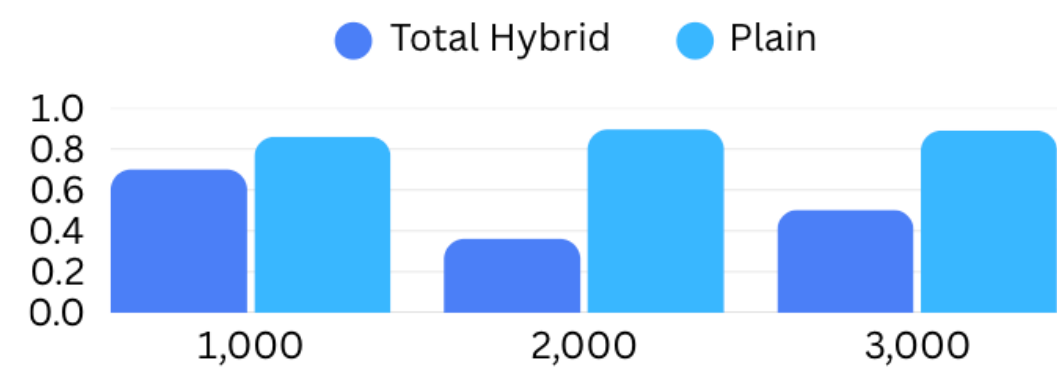
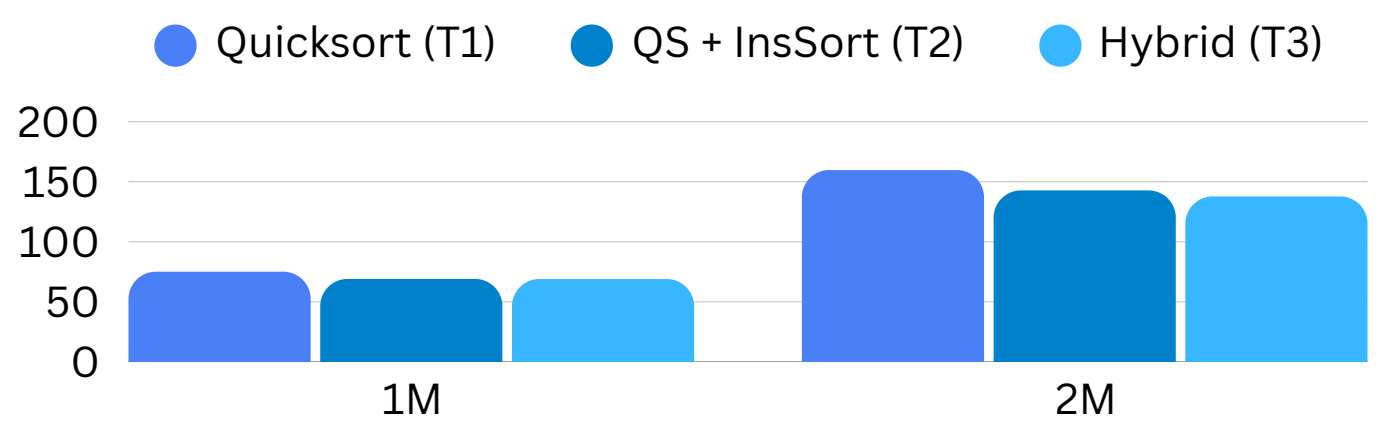


Table 3 - Algorithm Speed Comparison

Expected : Proposed Hybrid < QS + InsSort < Classic QS

n = r	Quicksort (T1)	QS + InsSort (T2)	Hybrid (T3)
1M	75.1	69.08	68.98
2M	159.78	142.83	137.82



Discussion & Analysis

Table 1 - Cache Locality Confirmed

Speedups of 1.35x, 1.73x, 2.49x confirm that data locality has a real and scalable impact on Counting Sort performance. The trend aligns with the paper's 2-3x prediction. Differences in our values and paper values are attributed to differences in hardware, both machines confirm the same fundamental effect.

Table 2 - Preprocessing Overhead is Negligible

The hybrid consistently outperformed plain Counting Sort in all cases. Preprocessing overhead remained less than 0.08ms across all trails. This directly validates the paper's claim that partitioning into cache sized sub arrays before sorting is worthwhile even when $r > n$.

Table 3 - Ordering Preserved

Our results matched the paper's ordering exactly Hybrid < QS + InsSort < Classic QS. The gap widening from 8% to 14% confirms that the cache locality advantage scales with input size. The smaller ratio versus the paper's 25% is explained by hardware, our M1 chip has lower classical ds times than the papers values suggesting faster single core performance.

Key Numerical Results

8% faster

than Quicksort
 $n = r = 1,000,000$

14% faster

than Quicksort
 $n = r = 2,000,000$

2.49x

max speedup
sorted vs unsorted input

T1 < T2 < T3

ordering preserved
both input sizes

Conclusion

The hybrid algorithm succeeds in its core objective. preprocessing into cache sized partitons reduces cache misses during the Counting sort phase, producing consistent performance gains that grow with input size. results across all three experimental tables validate the paper's findings.

Future Work

- Investigate stable preprocessing alternatives to restore sorting stability
- Develop adaptive threshold C selection based on runtime CPU cache detection
- Extend benchmarking to larger input sizes